

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

PHASE 2 — QUALITY & DEPLOYMENT



Session 15

Kubernetes Core

Pods · Liveness & Readiness Probes · Services · ConfigMap & Secrets · Namespaces



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Wednesday

TIME

3:00–5:30 PM

DURATION

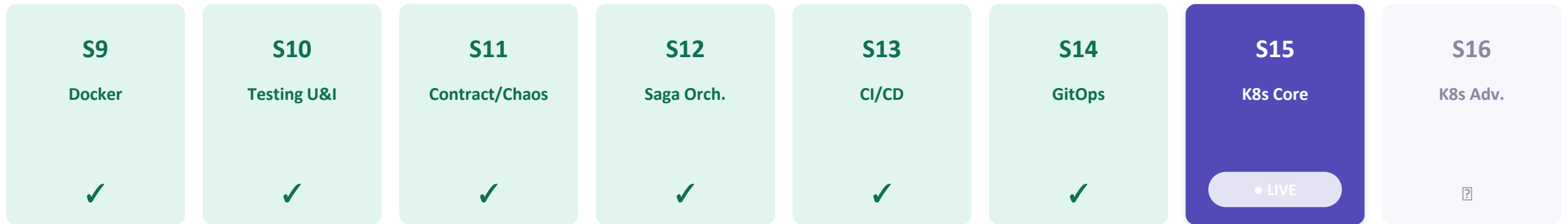
2.5 Hours

PHASE

Phase 2 · Session 15/29

Where We Are — Phase 2

Foundation & Core Patterns (Phase 1) is complete. Today: Session 15 of 8 in Phase 2 — Quality & Deployment.



Full 29-session roadmap lives only in the Kickoff deck. This is Phase 2's context — Sessions 9 through 16.

S14 introduced K8s manifests as a light intro for ArgoCD context. S15 goes deeper: probes, service types, Secrets, and namespaces make the deployment production-ready.

From "It Deploys" to "It's Production-Ready"



S14 gave us

ArgoCD deploys product-service. Git manifests exist: Deployment, Service, ConfigMap.



S14 was intro-only

Explicit scope boundary: Ingress, RBAC, Secrets, HPA — all deferred to S15/S16.



S15 fills the gap

K8s has no health signal from our app yet. No probes. DB password sits in plain ConfigMap.

Today's framing: Kubernetes has no idea our app is healthy unless we tell it how to ask. That's the whole session.

Pod Is "Running" — But Not Serving Traffic

- 1 We deployed product-service to Kubernetes. `kubectl get pods` shows: Running.
- 2 We curl the product endpoint through the Service. Timeout. 503.
- 3 The pod is Running — but it is NOT ready. Spring Boot is still loading JPA schemas, connecting to PostgreSQL, registering with Eureka, warming up the cache.
- 4 **Kubernetes sent traffic to the pod before it finished starting. The first 30 requests fail.**

🗣️ Ask the class: "How many of you have seen a service that appears healthy but stops responding?"

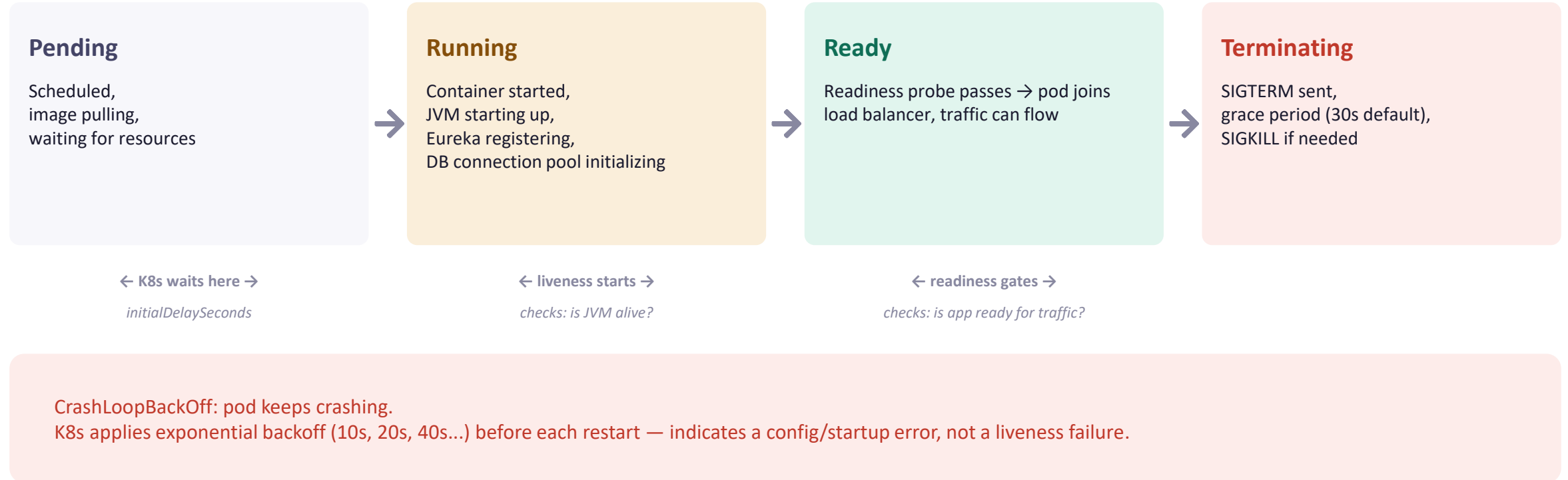
A Silent Deadlock — Running, But Dead Inside

- A pod has been running for 6 hours.
- It silently entered a deadlock — JVM threads all blocked, process still exists, JVM still consuming CPU.
- `kubectl get pods`: Running.
- Customers get timeouts. The pod never returns errors — it just hangs forever.

Both scenarios have the same root cause: Kubernetes has no health signal from the application.

Liveness probe fixes deadlocks. Readiness probe fixes premature traffic. Both are required.

Pod Lifecycle



Why a Pod, Not a Container?



Docker Container

A running process with its own filesystem and network namespace.
The unit Docker manages directly.



Kubernetes Pod

K8s' deployable unit. Wraps one or more containers that share network/storage.
You never run containers directly in K8s — you declare Pods (via Deployments).

Liveness vs Readiness Probe

	Liveness Probe: 'Is this pod still alive?'	Readiness Probe: 'Is this pod ready to accept traffic?'
If it FAILS	RESTART the pod (kills it and starts a new one)	REMOVE from load balancer (pod keeps running)
Failure scenario	Deadlock, OOM, Infinite Loops, JVM freeze	Spring Boot still starting, warm-up cache loading, DB migration in progress
Spring Boot endpoint	/actuator/health/liveness	/actuator/health/readiness
DB connectivity check?	☐ No — DB outage ≠ pod is dead	☐ Yes — if DB is down, pod should not serve traffic

COMMON TRAP: "I only need one probe — liveness is enough." → They serve different purposes. Configure BOTH.

CORRECT: They serve different purposes. Configure BOTH. Missing readiness = premature traffic.

Spring Boot Actuator — Health Sub-Endpoints

```
# Spring Boot Actuator config (application.yml):  
  
management:  
  endpoints:  
    web:  
      exposure:  
        include: health,info  
  health:  
    livenessstate:  
      enabled: true      # enables /actuator/health/liveness  
    readinessstate:  
      enabled: true      # enables /actuator/health/readiness
```

❗ Both endpoints require spring-boot-starter-actuator in pom.xml AND the livenessstate/readinessstate flags enabled. Without both, the sub-health endpoints do not exist — probes will 404.

Rule: never add DB or Redis checks to the liveness group — a DB outage should not restart the pod. DB checks belong in readiness.

product-service Deployment — Probe Configuration

In `services/product-service Deployment spec.containers[0]`:

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8081
  initialDelaySeconds: 60    # Spring Boot needs time to start. For DEV inly: reduce to 30 for faster feedback
  periodSeconds: 10         # check every 10 seconds
  failureThreshold: 3       # restart after 3 consecutive failures

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8081
  initialDelaySeconds: 30    # readiness checked earlier — pod might be alive before ready
  periodSeconds: 5          # check every 5 seconds
  failureThreshold: 3       # remove from LB after 3 failures
```

Verifying Probes Are Wired Correctly

```
$ kubectl describe pod -l app=product-service -n ecommerce
```

```
Liveness:  http-get http://:8081/actuator/health/liveness delay=60s  
           timeout=1s period=10s #success=1 #failure=3
```

```
Readiness: http-get http://:8081/actuator/health/readiness delay=30s  
           timeout=1s period=5s #success=1 #failure=3
```

Events:

Normal	Started	12s	kubelet	Started container product-service
Normal	Pulled	15s	kubelet	Successfully pulled image
Warning	Unhealthy	8s	kubelet	Readiness probe failed: HTTP 503

🔗 Show it live: the Warning line disappears once Spring Boot finishes startup — readiness flips to Healthy and the pod joins the Service endpoints.

Kubernetes Service Type: ClusterIP vs NodePort vs LoadBalancer

Internal service-to-service call only?

→ **ClusterIP** (default — no external access)

Testing externally on dev cluster (minikube)?

→ **NodePort** (random port 30000-32767)

Production external access (cloud provider)?

→ **LoadBalancer** (cloud creates L4 load balancer)

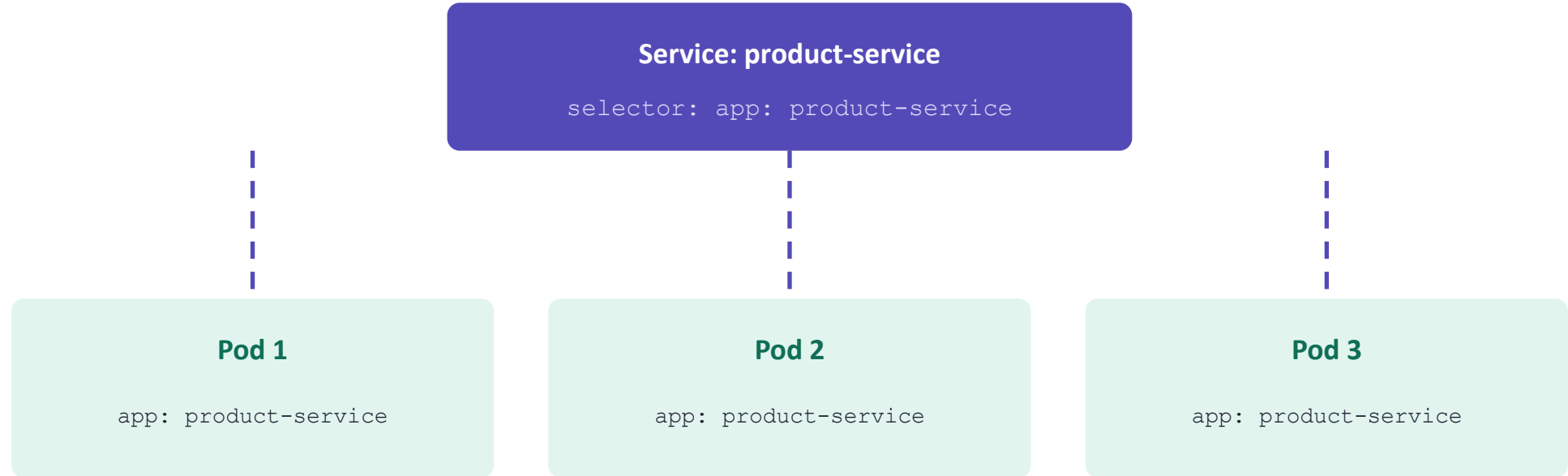
HTTP routing with host/path rules?

→ **Ingress + ClusterIP** (beyond today's scope)

Database or internal cache (Redis/Kafka)?

→ **ClusterIP only** — never expose externally

How a Service Finds Its Pods



The Service **does not** track pod IPs directly — it continuously watches for pods whose **labels** match its **selector**.
Add a 4th pod with the same label and it automatically joins.
Delete one and it's automatically removed.
This is why the **label selector** is the single most important line in a **Service** manifest.

Kubernetes Service Type: ClusterIP vs NodePort vs LoadBalancer

```
# ClusterIP (default) — internal only
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: ecommerce
spec:
  selector:
    app: product-service
  ports:
    - port: 8081
      targetPort: 8081
  type: ClusterIP      # default — omit type: for same result
# NodePort — for local dev / testing
# type: NodePort
# ports:
#   - port: 8081
#     targetPort: 8081
#     nodePort: 30081  # optional — K8s assigns random if omitted
# DEV ONLY: NodePort exposes on every node's IP. Never use in production.
```

ConfigMap vs Secret

Rule: if you would be embarrassed to see it in a git log, it belongs in a Secret, not a ConfigMap.

	ConfigMap	Secret
Data type	Non-sensitive configuration	Passwords, tokens, API keys, certificates
Storage	Plain text in etcd	Base64-encoded in etcd (NOT encrypted by default)
In git?	 Safe to commit	 Never commit — use external secrets manager in prod
Example	datasource.url, redis.host, eureka.url	DB_PASSWORD, JWT_SECRET, DOCKER_TOKEN

Secret for the DB Password

Secret for product-service DB password

```
apiVersion: v1
kind: Secret
metadata:
  name: product-secrets
  namespace: ecommerce
type: Opaque
data:
  DB_PASSWORD: cG9zdGdyZXNwYXNzd29yZA== # base64('postgrespassword')
# DEV ONLY: hardcoded base64. Prod: Sealed Secrets / Vault / AWS Secrets Manager.

# Deployment references it:
env:
- name: SPRING_DATASOURCE_PASSWORD
  valueFrom:
    secretKeyRef: {name: product-secrets, key: DB_PASSWORD}
```

Namespace Isolation — One Per Environment

`ecommerce-dev`

Developer testing

Lower resource limits

`ecommerce`

Integration / staging

What we use in this course

`ecommerce-prod`

Production

Stricter limits, manual sync policy

Namespaces provide soft multi-tenancy within a single cluster: resource quotas, RBAC, and network policies per namespace.
kubectl commands default to the 'default' namespace — always specify -n ecommerce or set the context.

Creating & Using the Namespace

```
# Create ecommerce namespace
kubectl create namespace ecommerce

# Or via ArgoCD Application syncOptions:
# syncOptions:
#   - CreateNamespace=true

# View all resources in the namespace:
kubectl get all -n ecommerce

# Set default context so you stop typing -n every time:
kubectl config set-context --current --namespace=ecommerce
```

Production-Ready product-service in Kubernetes

25 min in-session + homework · Builds on S14 manifests · Grading: Feature 70% + Tests 20% + Quality 10%



Update Spring Boot Actuator Config

Enable livenessstate + readinessstate in application.yml (3 min)



Add Probes to Deployment YAML

livenessProbe + readinessProbe with correct delays (8 min)



Create Secret + Reference It

product-secrets Secret, wire via secretKeyRef (10 min)



Verify and Commit

kubectl describe pod, test endpoints, git push (4 min)

Acceptance Criteria

- ✓ `/actuator/health/liveness` and `/actuator/health/readiness` return HTTP 200
- ✓ Deployment YAML has both `livenessProbe` and `readinessProbe` configured
- ✓ Secret 'product-secrets' exists in namespace `ecommerce`
- ✓ Deployment references `DB_PASSWORD` from `secretKeyRef` (not `ConfigMap`)
- ✓ `kubectl describe pod` shows probe status: `Liveness` and `Readiness` defined
- ✓ ArgoCD Application status: `Synced + Healthy` after probe addition
- ✓ Commit: `session-15: add-probes-and-secret-product-service`

BONUS: Add `startupProbe` (`failureThreshold: 30`) for slow startup + order-service probes (same pattern)

What We Added Today

 Capability Added

Production-Ready K8s Deployment

K8s now trusts health signals from the app: liveness restarts on deadlock, readiness prevents premature traffic, Secret keeps DB password out of ConfigMap.

 Platform Updates

k8s/product-service/deployment.yaml — probes added
k8s/product-service/secret.yaml — new (DEV base64)
application.yml — livenessstate + readinessstate enabled
ArgoCD synced with production-ready config

Enterprise E-Commerce Platform — After Session 15

- S9 
- S10-11 
- S12 
- S13 
- S14 
- S15 
- S16 

5 Errors You Will See This Session

Pod stuck in CrashLoopBackOff

Increase initialDelaySeconds — Spring Boot + JPA needs 60-90s. Check kubectl logs --previous.

/actuator/health/liveness returns 404

Need BOTH spring-boot-starter-actuator AND management.health.livenessstate.enabled: true

Secret values corrupted

Use echo -n 'pw' | base64 — the -n flag avoids a trailing newline corrupting the decoded value

ReadinessProbe passes but no traffic

Check Service selector labels match Pod labels exactly — one typo = zero endpoints

Running but ArgoCD shows Degraded

ArgoCD uses readiness for Health. Ensure readiness probe passes before sync completes



SESSION 15 COMPLETE

Production-Ready K8s Deployment

Next: Session 16 — Kubernetes Advanced

HPA · Resource Limits · Helm Charts · RBAC · Phase 2 Retrospective

Monday, 3:00–5:30 PM, Online — Phase 2 Final Session